

佛山科学技术学院

课程设计说明书

设计名称：机器学习

题目：基于卷积神经网络的图像分类

学生姓名：温硕勋

专业：智能科学与技术

班级：21 智能科学与技术 1 班

学号：

时间：2023 年 12 月 25 日至 2024 年 1 月 19 日

电子信息工程学院

目录

一、问题分析.....	1
二、程序设计与实现.....	1
2.1 程序逻辑与关键组件.....	1
2.1.1 卷积神经网络结构.....	1
2.1.2 卷积神经网络模型编译.....	2
2.1.3 卷积神经网络模型的训练.....	2
2.1.4 可视化卷积神经网络的训练过程，即损失函数值的变化曲线.....	3
2.1.5 卷积神经网络模型评估.....	3
2.1.6 卷积神经网络模型测试.....	3
2.1.7 设置不同的超参数值，观察、记录与分析实验结果.....	4
2.1.8 变化卷积神经网络中的卷积层、池化层的个数，观察、记录与分析实验结果.....	4
2.1.9 替换卷积神经网络中的损失函数为平方损失函数，观察、记录与分析实验结果.....	4
2.1.10 将得到的卷积神经网络 model 应用于 MNIST 手写数字识别问题.....	4
2.2 关键源代码.....	5
2.2.1 用 CIFAR-10 数据集训练并测试的卷积神经网络模型.....	5
2.2.2 改变超参数，三层卷积层的 filters 分别改为 16、32、16，池化层不变：.....	8
2.2.3 改变超参数，三层卷积层的 filters 分别改为 32、64、64，池化层不变：.....	9
2.2.4 改变卷积层、池化层的层数，改为两层，池化层参数不变，卷积层 filter 分别为 16、32，内核为 2：.....	10
2.2.5 改变卷积层、池化层的层数，改为四层，池化层参数不变，卷积层 filter 分别为 32、64、32、64，内核为 4：.....	11
2.2.6 在卷积层、池化层的层数为四层，池化层参数不变，卷积层 filter 分别为 32、64、32、64，内核为 4 的情况下，将训练次数翻倍：.....	12
2.2.7 用 MNIST 数据集训练并测试的卷积神经网络模型.....	14
三、个人心得体会.....	21

一、问题分析

在机器学习课程设计中，我们需要构建的卷积神经网络模型，用于对 CIFAR-10 数据集中的数据进行分类。在数据输入到卷积神经网络模型之前，需要对数据进行归一化、去均值或 PCA，而具体使用哪种方法，应根据实际要求进行处理。在卷积神经网络模型中的输入层还要给定输入数据的正确格式，包括图片大小和通道数。随后是对卷积层和池化层的设计，在初步设置卷积层和池化层均为 3 层，并在后续的实验对比中测试出较好的层数，在卷积层中 filter 的设置定为 8、16、32、64 等，根据后续实践结果分析后，选择过滤效果较好的来使用。内核大小也在 2*2、3*3 或 4*4 里对比选择，在卷积层中决定不对图片进行缩小，仅在池化层进行对图片的压缩，padding 超参数选择 same，激活函数选择简单快速的 ReLu，步长默认为 1。每一层卷积后跟一层池化层，池化层选用 MaxPooling 最大池化层，池化层窗口采用 2*2，并取 stride=2。随后跟一层全连接层和输出层，因为 CIFAR-10 数据集有十个类别，所以输出层最后输出数量为 10。

对模型的训练过程，初步制定是全部数据集输入训练，训练次数至少为1000。但由于电脑是在 2020 年购买，当时配置未选用最高，目前的设备算力有限，最后仅选择 10000 张图片进行训练，而且训练的次数也进行降低，在此情况下保证了训练的顺利完成，这也导致了模型的准确率有所降低。

二、程序设计与实现

2.1 程序逻辑与关键组件

2.1.1 卷积神经网络结构

1 层输入，3 层卷积层，3 层池化层，1 层全连接，1 层输出

```
model =tf.keras.Sequential([ tf.keras.Input(shape=(32,32,3)),  
tf.keras.layers.Conv2D(filters=32 ,kernel_size=3 , padding='same',  
activation='relu'),  
tf.keras.layers.MaxPooling2D(pool_size=(2,2),strides=(2,2)),  
tf.keras.layers.Conv2D(filters=64,kernel_size=3,padding='same', activation='relu'),  
tf.keras.layers.MaxPooling2D(pool_size=(2,2),strides=(2,2)),
```

```
tf.keras.layers.Conv2D(filters=32 ,kernel_size=3 , padding='same',
    activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=(2,2),strides=(2,2)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(10, activation='softmax')
```

Layer (type)	Output Shape	Param #
conv2d_8 (Conv2D)	(None, 32, 32, 32)	896
max_pooling2d_8 (MaxPooling2)	(None, 16, 16, 32)	0
conv2d_9 (Conv2D)	(None, 16, 16, 64)	18496
max_pooling2d_9 (MaxPooling2)	(None, 8, 8, 64)	0
conv2d_10 (Conv2D)	(None, 8, 8, 32)	18464
max_pooling2d_10 (MaxPooling)	(None, 4, 4, 32)	0
flatten_3 (Flatten)	(None, 512)	0
dense_3 (Dense)	(None, 10)	5130

```
)
```

2.1.2 卷积神经网络模型编译

```
model.compile(
    optimizer = tf.keras.optimizers.Adam(
    learning_rate=0.001),
    loss = tf.keras.losses.sparse_categorical_crossentropy,
    metrics = ['acc']
)
```

2.1.3 卷积神经网络模型的训练

```
batches = 50    #训练次数  batch_size = 10000    #抽取图片  data = []
for i in range(batches):
    index=np.random.choice(x.shape[0],batch_size,replace=False)
    x_batch = x[index]
```

```
y_batch = y[index]
history = model.fit(x_batch,y_batch, batch_size=1000, validation_split=0.2)
hist = pd.DataFrame(history.history)
data.append(hist)
df = pd.concat(data).reset_index(drop=True)    # 竖着合并
```

2.1.4 可视化卷积神经网络的训练过程，即损失函数值的变化曲线

```
def plot_history(hist):
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.xlabel('Epoch')
plt.plot(hist['loss'],
label='loss')
plt.plot(hist['val_loss'],
label='val_loss')
plt.legend()
plt.subplot(1, 2, 2)
plt.xlabel('Epoch')
plt.plot(hist['acc'],
label='acc', color='red')
plt.plot(hist['val_acc'],
label='val_acc') plt.legend()
plt.show()
plot_history(df)
```

2.1.5 卷积神经网络模型评估

```
stiur=model.evaluate(x_test, y_test, verbose=2)
print('模型评估数值: ',stiur)
```

2.1.6 卷积神经网络模型测试

```
prediction = model.predict(x_test) class_names = [1,2,3,4,5,6,7,8,9,10] nur=0
```

```
for i in range(1000):  
    pre = class_names[np.argmax(prediction[i])] tar = class_names[y_test[i][0]]  
    if (pre == tar):  
        nur+=1  
  
print('模型预测准确率为: ',nur*100/1000,'%')
```

2.1.7 设置不同的超参数值，观察、记录与分析实验结果

(1)改变超参数，三层卷积层的 filters 分别改为 16、32、16，池化层不变。
具体见 2.2.2;

(2)改变超参数，三层卷积层的 filters 分别改为 32、64、64，池化层不变。
具体见 2.2.3;

2.1.8 变化卷积神经网络中的卷积层、池化层的个数，观察、记录与分析实验结果

(1)改变卷积层、池化层的层数，改为两层，池化层参数不变，卷积层 filter 分别为 16、32，内核为 2，具体见 2.2.4;

(2) 改变卷积层、池化层的层数，改为四层，池化层参数不变，卷积层 filter 分别为 32、64、32、64，内核为 4，具体见 2.2.5。

2.1.9 替换卷积神经网络中的损失函数为平方损失函数，观察、记录与分析实验结果

在卷积层、池化层的层数为四层，池化层参数不变，卷积层 filter 分别为 32、64、32、64，内核为 4 的情况下，将训练次数翻倍，具体见 2.2.6。

2.1.10 将得到的卷积神经网络model应用于MNIST手写数字识别问题

具体见 2.2.7。

2.2 关键源代码

2.2.1 用 CIFAR-10 数据集训练并测试的卷积神经网络模型

```
import tensorflow as tf

(x, y), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()

import pandas as pd import numpy as np

import matplotlib.pyplot as plt

x = x/255    #归一化 x_test = x_test/255

"""模型搭建"""

model=tf.keras.Sequential([ tf.keras.Input(shape=(32,32,3)),

tf.keras.layers.Conv2D(filters=32      ,kernel_size=3      ,      padding='same',

activation='relu'), tf.keras.layers.MaxPooling2D(pool_size=(2, 2),strides=(2,2)),

tf.keras.layers.Conv2D(filters=64      ,kernel_size=3      ,      padding='same',

activation='relu'),

tf.keras.layers.MaxPooling2D(pool_size=(2,          2),strides=(2,2)),

tf.keras.layers.Conv2D(filters=32      ,kernel_size=3      ,      padding='same',

activation='relu'), tf.keras.layers.MaxPooling2D(pool_size=(2, 2),strides=(2,2)),

tf.keras.layers.Flatten(),

tf.keras.layers.Dense(10, activation='softmax')

])

model.summary()

"""模型编译"""

model.compile(

optimizer      =      tf.keras.optimizers.Adam(learning_rate=0.001),      loss      =

tf.keras.losses.sparse_categorical_crossentropy, metrics = ['acc']

)
```

```
"""模型训练"""

batches = 50    #训练次数

batch_size = 10000 #抽取图片

data = []

for i in range(batches):

    index=np.random.choice(x.shape[0],batch_size,replace=False)

    x_batch = x[index]

    y_batch = y[index]

    history = model.fit(x_batch,y_batch, batch_size=1000, validation_split=0.2)

    hist = pd.DataFrame(history.history)

    data.append(hist)

df = pd.concat(data).reset_index(drop=True)    # 竖着合并

# 损失函数可视化

def plot_history(hist):

    plt.figure(figsize=(10, 5))

    plt.subplot(1, 2, 1)

    plt.xlabel('Epoch')

    plt.plot(hist['loss'],

    label='loss')

    plt.plot(hist['val_loss'],

    label='val_loss') plt.legend()

    plt.subplot(1, 2, 2)

    plt.xlabel('Epoch')

    plt.plot(hist['acc'],

    label='acc', color='red')

    plt.plot(hist['val_acc'],

    label='val_acc')

    plt.legend()

    plt.show()
```

```
plot_history(df)

"""模型评估"""

stiur=model.evaluate(x_test, y_test, verbose=2)

print('模型评估数值: ',stiur)

"""模型预测"""

prediction = model.predict(x_test)

class_names = [1,2,3,4,5,6,7,8,9,10]

nur=0

for i in range(1000):

pre = class_names[np.argmax(prediction[i])]

tar = class_names[y_test[i][0]]

if (pre == tar):

nur+=1

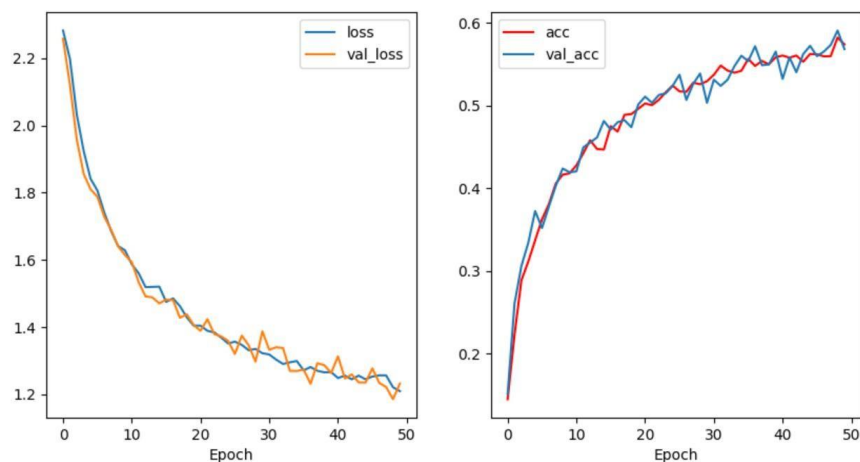
print('模型预测准确率为: ',nur*100/1000,'%')
```

训练结果测试图:

模型结构

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 32, 32, 32)	896
max_pooling2d (MaxPooling2D)	(None, 16, 16, 32)	0
conv2d_1 (Conv2D)	(None, 16, 16, 64)	18496
max_pooling2d_1 (MaxPooling2D)	(None, 8, 8, 64)	0
conv2d_2 (Conv2D)	(None, 8, 8, 32)	18464
max_pooling2d_2 (MaxPooling2D)	(None, 4, 4, 32)	0
flatten (Flatten)	(None, 512)	0
dense (Dense)	(None, 10)	5130

损失函数变化图



```

8/8 [=====] - 27s 3s/step - loss: 1.2
8/8 [=====] - 56s 6s/step - loss: 1.2
313/313 - 16s - loss: 1.2237 - acc: 0.5767 - 16s/epoch - 50ms/
模型评估数值: [1.2237221002578735, 0.57669997215271]
313/313 [=====] - 11s 9ms/step
模型预测准确率为: 58.6 %

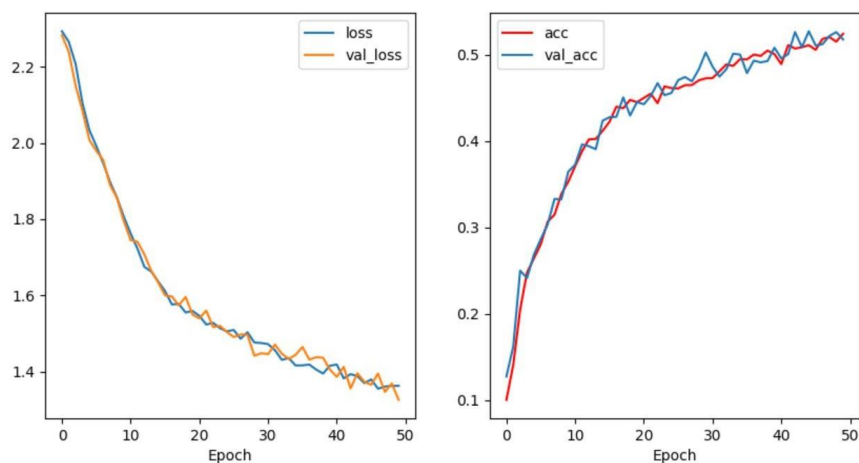
```

2.2.2 改变超参数，三层卷积层的 filters 分别改为 16、32、16，池化层不变：

模型结构

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 32, 32, 16)	448
max_pooling2d (MaxPooling2D)	(None, 16, 16, 16)	0
conv2d_1 (Conv2D)	(None, 16, 16, 32)	4640
max_pooling2d_1 (MaxPooling2D)	(None, 8, 8, 32)	0
conv2d_2 (Conv2D)	(None, 8, 8, 16)	4624
max_pooling2d_2 (MaxPooling2D)	(None, 4, 4, 16)	0
flatten (Flatten)	(None, 256)	0
dense (Dense)	(None, 10)	2570

损失函数变化图



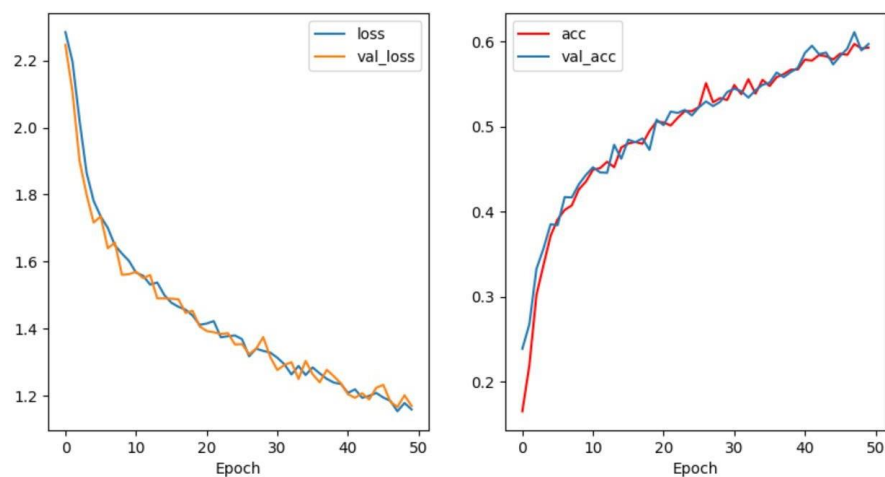
```
8/8 [=====] - 51s 7s/step - loss: 1.3628 -
313/313 - 8s - loss: 1.3591 - acc: 0.5186 - 8s/epoch - 26ms/step
模型评估数值: [1.3591183423995972, 0.5185999870300293]
313/313 [=====] - 11s 13ms/step
模型预测准确率为: 51.5 %
```

2.2.3 改变超参数，三层卷积层的 filters 分别改为 32、64、64，池化层不变：

模型结构

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 32, 32, 32)	896
max_pooling2d (MaxPooling2D)	(None, 16, 16, 32)	0
conv2d_1 (Conv2D)	(None, 16, 16, 64)	18496
max_pooling2d_1 (MaxPooling2D)	(None, 8, 8, 64)	0
conv2d_2 (Conv2D)	(None, 8, 8, 64)	36928
max_pooling2d_2 (MaxPooling2D)	(None, 4, 4, 64)	0
flatten (Flatten)	(None, 1024)	0
dense (Dense)	(None, 10)	10250

损失函数变化图



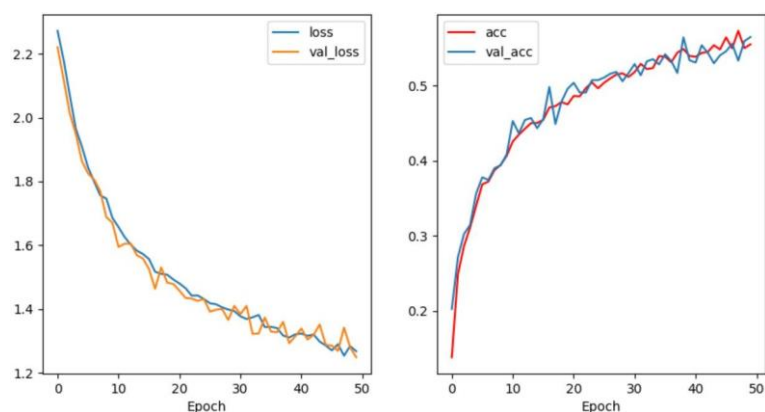
```
313/313 - 15s - loss: 1.2217 - acc: 0.5684 - 15s/epoch - 49ms/step
模型评估数值: [1.2217043638222937, 0.5684000253677368]
313/313 [=====] - 5s 9ms/step
模型预测准确率为: 55.7 %
```

2.2.4 改变卷积层、池化层的层数，改为两层，池化层参数不变，卷积层 **filter** 分别为 16、32，内核为 2:

模型结构

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 32, 32, 16)	208
max_pooling2d (MaxPooling2D)	(None, 16, 16, 16)	0
conv2d_1 (Conv2D)	(None, 16, 16, 32)	2080
max_pooling2d_1 (MaxPooling2D)	(None, 8, 8, 32)	0
flatten (Flatten)	(None, 2048)	0
dense (Dense)	(None, 10)	20490
Total params: 22,778		
Trainable params: 22,778		
Non-trainable params: 0		

损失函数变化图



```

313/313 - 5s - loss: 1.2822 - acc: 0.5545 - 5s/epoch - 17ms/step
模型评估数值: [1.282223105430603, 0.5544999837875366]
313/313 [=====] - 3s 4ms/step
模型预测准确率为: 55.5 %

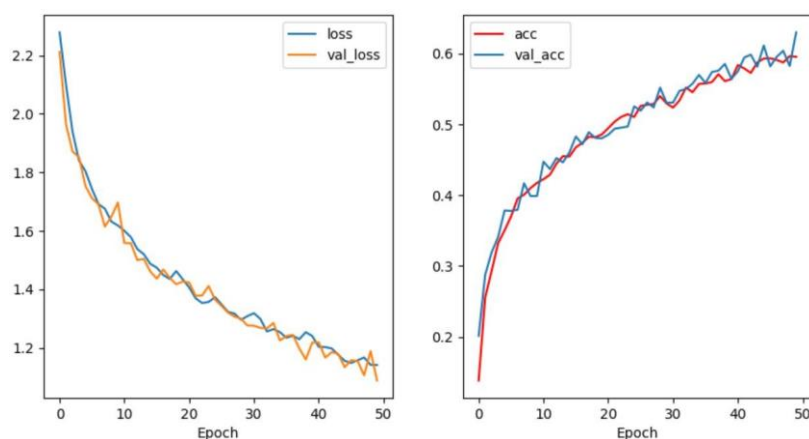
```

2.2.5 改变卷积层、池化层的层数，改为四层，池化层参数不变，卷积层 **filter** 分别为 32、64、32、64，内核为 4:

模型结构

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 32, 32, 32)	1568
max_pooling2d (MaxPooling2D)	(None, 16, 16, 32)	0
conv2d_1 (Conv2D)	(None, 16, 16, 64)	32832
max_pooling2d_1 (MaxPooling2D)	(None, 8, 8, 64)	0
conv2d_2 (Conv2D)	(None, 8, 8, 32)	32800
max_pooling2d_2 (MaxPooling2D)	(None, 4, 4, 32)	0
conv2d_3 (Conv2D)	(None, 4, 4, 64)	32832
max_pooling2d_3 (MaxPooling2D)	(None, 2, 2, 64)	0
flatten (Flatten)	(None, 256)	0
dense (Dense)	(None, 10)	2570

损失函数变化图



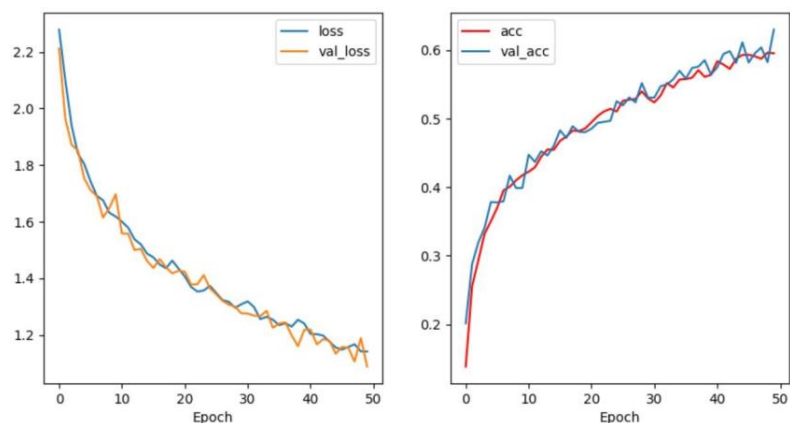
```
313/313 - 10s - loss: 1.1529 - acc: 0.5912 - 10s/epoch - 33ms/step
模型评估数值: [1.1529439687728882, 0.5911999940872192]
313/313 [=====] - 13s 12ms/step
模型预测准确率为: 60.3 %
```

2.2.6 在卷积层、池化层的层数为四层，池化层参数不变，卷积层 **filter** 分别为 32、64、 32、64，内核为 4 的情况下，将训练次数翻倍：

模型结构

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 32, 32, 32)	1568
max_pooling2d (MaxPooling2D)	(None, 16, 16, 32)	0
conv2d_1 (Conv2D)	(None, 16, 16, 64)	32832
max_pooling2d_1 (MaxPooling2D)	(None, 8, 8, 64)	0
conv2d_2 (Conv2D)	(None, 8, 8, 32)	32800
max_pooling2d_2 (MaxPooling2D)	(None, 4, 4, 32)	0
conv2d_3 (Conv2D)	(None, 4, 4, 64)	32832
max_pooling2d_3 (MaxPooling2D)	(None, 2, 2, 64)	0
flatten (Flatten)	(None, 256)	0
dense (Dense)	(None, 10)	2570

损失函数变化图



```
313/313 - 6s - loss: 1.0434 - acc: 0.6321 - 6s/epoch - 18ms/step
模型评估数值: [1.043393850326538, 0.632099986076355]
313/313 [=====] - 4s 11ms/step
模型预测准确率为: 64.3 %
```

实验结果分析:

通过仅改变卷积层超参数的结果对比得出, 卷积层超参数 `filter` 的选择在 32 和 64 能得出比较好的模型性能, 而且要经过所有的卷积层和池化层出来后的图片尺寸不能和过滤器的数量设置相差过大, 不然也会造成训练模型的性能下降。

另外, 发现在卷积层超参数 `filter` 取较小数值如 16 或 8 时减少卷积层和池化层的数量同时, 也减少卷积核大小, 训练出模型的性能反而有小幅度的提升。但总体数据实践对比表明, 卷积层和池化层的数量适当增加是有利于提升模型的性能, 而且模型的训练次数越多, 得到的模型性能越好, 但由于电脑的算力和存储满足不了大训练的条件, 只能通过小幅度对比了。

2.2.7 用 MNIST 数据集训练并测试的卷积神经网络模型

```
import tensorflow as tf
import numpy as np

def load_data(path='mnist.npz'):
    origin_folder = 'https://storage.googleapis.com/tensorflow/tf-keras-datasets/'
    path = tf.keras.utils.get_file(
        path,
        origin=origin_folder + 'mnist.npz',
        cache_dir='D:\\学习资料\\大三\\机器学习\\实践周\\基于 Tensorflow 图像识别', cache_subdir=""
    )

    with np.load(path, allow_pickle=True) as f: x_train, y_train = f['x_train'], f['y_train']
    x_test, y_test = f['x_test'], f['y_test']
    return (x_train, y_train), (x_test, y_test)

(x, y), (x_test, y_test) = load_data(path='mnist.npz')

import pandas as pd
import matplotlib.pyplot as plt

x = x / 255 # 归一化 x_test = x_test/255

"""模型搭建"""
model = tf.keras.Sequential([tf.keras.Input(shape=(28, 28, 1)),
                             tf.keras.layers.Conv2D(filters=32, kernel_size=3,
padding='same', activation='relu'),
                             tf.keras.layers.MaxPooling2D(pool_size=(2, 2),
```

```
strides=(2, 2)),
                                tf.keras.layers.Conv2D(filters=64, kernel_size=3,
padding='same', activation='relu'),
                                tf.keras.layers.MaxPooling2D(pool_size=(2, 2),
strides=(2, 2)),
                                tf.keras.layers.Conv2D(filters=32, kernel_size=3,
padding='same', activation='relu'),
                                tf.keras.layers.MaxPooling2D(pool_size=(1, 1),
strides=(1, 1)), tf.keras.layers.Flatten(),
                                tf.keras.layers.Dense(10, activation='softmax')
                                ])

model.summary()
"""模型编译"""
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss=tf.keras.losses.sparse_categorical_crossentropy,
    metrics=['acc']
)

"""模型训练"""
batches = 100 # 训练次数 batch_size = 10000 #抽取图片 data = []
for i in range(batches): index = np.random.choice(x.shape[0], batch_size,
replace=False)
x_batch = x[index]
y_batch = y[index]
history = model.fit(x_batch, y_batch, batch_size=1000, validation_split=0.2) # 训
练
hist = pd.DataFrame(history.history)
```

```
data.append(hist)

df = pd.concat(data).reset_index(drop=True) # 竖着合并

# 损失函数可视化
def plot_history(hist): plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.xlabel('Epoch')
plt.plot(hist['loss'],
         label='loss')
plt.plot(hist['val_loss'],
         label='val_loss')
plt.legend()

plt.subplot(1, 2, 2)
plt.xlabel('Epoch')
plt.plot(hist['acc'],
         label='acc', color='red')

plt.plot(hist['val_acc'],
         label='val_acc')
plt.legend()
plt.show()

plot_history(df)
"""模型评估"""
stiur = model.evaluate(x_test, y_test, verbose=2)
print('模型评估数值: ', stiur)
"""模型预测"""
```

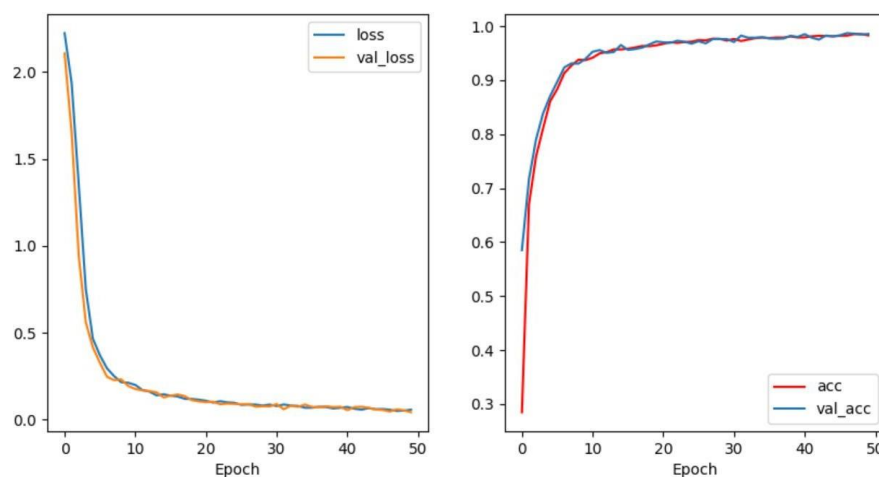
```
prediction = model.predict(x_test)
class_names = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
nur = 0
for i in range(1000):
    pre = class_names[np.argmax(prediction[i])]
    tar = class_names[y_test[i]]
    if (pre == tar):
        nur += 1
print('模型预测准确率为: ', nur * 100 / 1000, '%')
```

训练结果测试图:

模型结构

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 28, 28, 32)	320
max_pooling2d (MaxPooling2D)	(None, 14, 14, 32)	0
conv2d_1 (Conv2D)	(None, 14, 14, 64)	18496
max_pooling2d_1 (MaxPooling2D)	(None, 7, 7, 64)	0
conv2d_2 (Conv2D)	(None, 7, 7, 32)	18464
max_pooling2d_2 (MaxPooling2D)	(None, 3, 3, 32)	0
flatten (Flatten)	(None, 288)	0
dense (Dense)	(None, 10)	2890

损失函数变化图



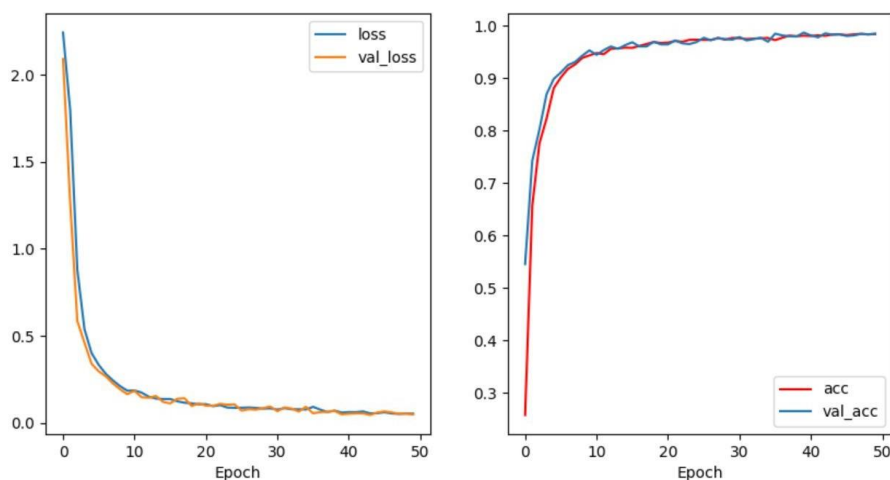
```
313/313 - 4s - loss: 0.0539 - acc: 0.9820 - 4s/epoch - 13ms/step
模型评估数值: [0.053881220519542694, 0.9819999933242798]
313/313 [=====] - 3s 7ms/step
模型预测准确率为: 98.1 %
```

减少一层池化层，增加一层卷积层，共四个卷积层，两个池化层：

模型结构

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 28, 28, 32)	320
max_pooling2d (MaxPooling2D)	(None, 14, 14, 32)	0
conv2d_1 (Conv2D)	(None, 14, 14, 64)	18496
max_pooling2d_1 (MaxPooling2D)	(None, 7, 7, 64)	0
conv2d_2 (Conv2D)	(None, 7, 7, 32)	18464
conv2d_3 (Conv2D)	(None, 7, 7, 16)	4624
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 10)	7850

损失函数变化图



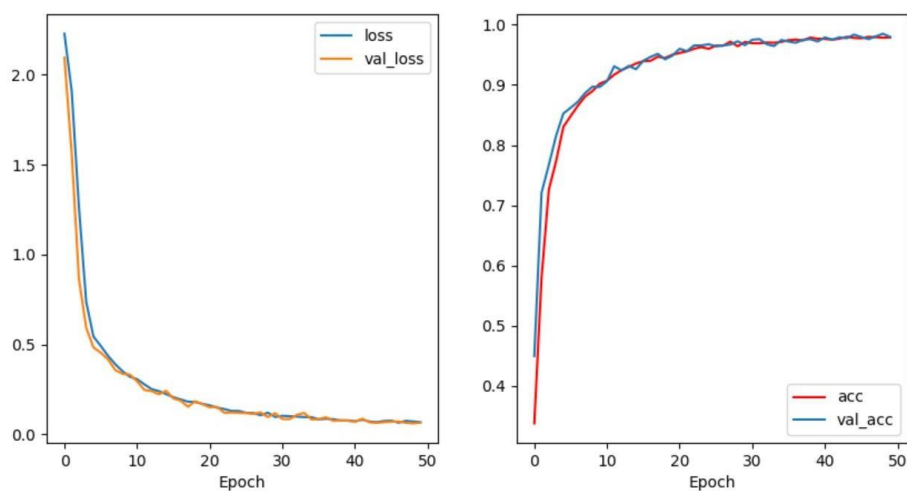
```
313/313 - 4s - loss: 0.0419 - acc: 0.9867 - 4s/epoch - 14ms/step
模型评估数值: [0.041855338960886, 0.9866999983787537]
313/313 [=====] - 3s 7ms/step
模型预测准确率为: 98.7 %
```

卷积层和池化层的层数保持不变，将最后一层池化层的窗口改为 1×1 ，既不对图片进行压缩：

模型结构

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 28, 28, 32)	160
max_pooling2d (MaxPooling2D)	(None, 14, 14, 32)	0
conv2d_1 (Conv2D)	(None, 14, 14, 64)	8256
max_pooling2d_1 (MaxPooling2D)	(None, 7, 7, 64)	0
conv2d_2 (Conv2D)	(None, 7, 7, 32)	8224
max_pooling2d_2 (MaxPooling2D)	(None, 7, 7, 32)	0
flatten (Flatten)	(None, 1568)	0
dense (Dense)	(None, 10)	15690

损失函数变化图



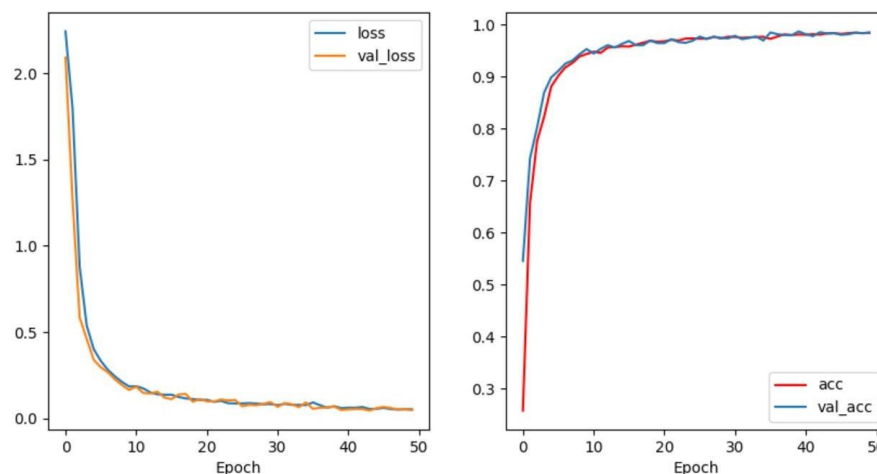
```
313/313 - 4s - loss: 0.0615 - acc: 0.9808 - 4s/epoch - 14ms/step
模型评估数值: [0.06145712360739708, 0.9807999730110168]
313/313 [=====] - 4s 7ms/step
模型预测准确率为: 98.3 %
```

保持卷积层和池化层数据不变，将模型训练次数翻倍：

模型结构

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 28, 28, 32)	320
max_pooling2d (MaxPooling2D)	(None, 14, 14, 32)	0
conv2d_1 (Conv2D)	(None, 14, 14, 64)	18496
max_pooling2d_1 (MaxPooling2D)	(None, 7, 7, 64)	0
conv2d_2 (Conv2D)	(None, 7, 7, 32)	18464
max_pooling2d_2 (MaxPooling2D)	(None, 7, 7, 32)	0
flatten (Flatten)	(None, 1568)	0
dense (Dense)	(None, 10)	15690

损失函数变化图



```
313/313 - 4s - loss: 0.0288 - acc: 0.9904 - 4s/epoch - 13ms/step
模型评估数值: [0.028841175138950348, 0.9904000163078308]
313/313 [=====] - 4s 6ms/step
模型预测准确率为: 99.0 %
```

实验结果分析:

将训练 CIFAR-10 数据集得到的卷积神经网络 model 应用于 MNIST 手写数字识别后,发现在同样的卷积神经网络模型情况下,训练得到的模型对 MNIST 数据集有很好的性能体现,可能原因是输入层的数据通道影响,通道越大证明数据集的复杂程度越大,对模型的构建要求更高。

在对 MNIST 数据集的训练上,对卷积神经网络模型的修改,增加了一层卷积层发现可以增加模型的训练效果,池化层可以减少一层,但如果减少窗口的大小反而会对模型训练起到反作用,目前在 MNIST 数据集上的卷积神经网络模型效果较好的结构是四层卷积层,两层池化层,其他不变。

三、个人心得体会

这次的实践使我对机器学习中的卷积神经网络模型有了更深的理解,包括模型的训练方式和训练过程。卷积神经网络模型一般有五层结构,分别是输入层、卷积计算层、激励层、池化层、全连接层,在实践过程中,由于框架的不一,具体搭建的结构也有所改变,代码实现过程中激励层的作用也在卷积层里,卷积后经过激励函数再输出。

如何根据训练数据的性质设计出合理高效的模型是难点, 关乎到构建卷积神经网络模型里卷积层和池化层的设置, 设置为多少层, 每层卷积层的超参数设置, filter 为多少合适, 内核多大, 深度多少, 是否需要填充值, 激励函数选什么? 如果没有相应的设计方法, 也只能以一般形式进行初步设计, 然后不断进行实践调整, 得出目前的最优模型, 但也是局部最优了。

在本次课程设计过程中, 我尝试过用 Pytorch 框架搭建卷积神经网络模型, 但我所查阅的资料构建出来的模型, 受限于电脑配置太低, 哪怕将模型参数改小了, 训练次数改小了也不行, 配置太低, 运行中电脑卡顿, 最后会出现训练中断, 得不到结果。而将各参数值修改过小, 又会导致训练出来的模型性能非常差, 不具备参考意义, 所以后面转用 TensorFlow 框架来构建卷积神经网络模型, 但也要把训练次数和处理图片数量减少到一定程度才能运行, 所以也导致了训练出来的模型性能并没有达到最好。

最重要的还是我对卷积神经网络的理解加深了, 对于卷积层和池化层的构建, 暂时还没找到较为系统的方法, 还需多加学习。